

VisioLab

Sandbox Interactivo de Visión Artificial

Equipo de desarrollo

Integrante	Rol	Responsabilidad
Alejandro Ortíz Cortés	Project Manager & UX/UI	Gestión del proyecto y diseño de interfaz
Álvaro Romero Castro	Lead Computer Vision Engineer	Pipeline gráfico y filtros espaciales
Brayan Muñoz Pérez	ML / AI Developer	Integración YOLO y modelos de detección
Juan Camilo López Bustos	Backend & Data Engineer	Matrices, histogramas y rendimiento

1. Herramientas Utilizadas

El proyecto integra un stack de tecnologías open-source orientadas a procesamiento visual, visualización de datos y despliegue rápido de interfaces web interactivas.

1.1 Frontend e Interfaz Web

Herramienta	Categoría	Uso en el proyecto
Streamlit $\geq$ 1.30	Framework web	Construcción de toda la interfaz interactiva: tabs, sidebar, sliders, radio buttons y renderizado de imágenes y gráficos.
HTML / CSS (inline)	Estilos UI	Tema oscuro personalizado con tipografía Inter, tarjetas glass-morphism, badges y animaciones CSS para el header.
Plotly $\geq$ 5.18	Visualización	Histogramas interactivos, gráfico radar de entropía, scatter 3D de color, barras de percentiles y perfil de intensidad.

1.2 Procesamiento de Imagen

Herramienta	Categoría	Uso en el proyecto
OpenCV ≥ 4.8	Computer Vision	Filtros espaciales (blur, bordes, morfología), ajustes de color, cálculo de histogramas y mapas de diferencias.
NumPy ≥ 1.24	Computación numérica	Manipulación de matrices de píxeles, cálculo de estadísticas por canal y operaciones vectorizadas.
Pillow ≥ 10.0	I/O de imágenes	Lectura y conversión de imágenes subidas por el usuario desde el file uploader de Streamlit.
scikit-image ≥ 0.21	Métricas	Cálculo del índice SSIM (Structural Similarity Index) para comparación de calidad entre imagen original y procesada.

1.3 Machine Learning y Detección de Objetos

Herramienta	Categoría	Uso en el proyecto
Ultralytics YOLOv8	ML / Detección	Inferencia de objetos sobre imágenes filtradas. Modelos disponibles: Nano, Small y Medium. Dataset COCO (80 clases).
PyTorch (dep. YOLO)	Deep Learning	Backend de inferencia del modelo YOLO. Se usa a través de la API de Ultralytics sin acceso directo.

1.4 Análisis y Datos

Herramienta	Categoría	Uso en el proyecto
Pandas	Análisis de datos	Construcción de tablas de estadísticas por canal (media, std, percentiles, entropía) para <code>st.dataframe()</code> .
Python 3.11+	Lenguaje base	Lenguaje principal del proyecto. Se usan <code>dataclasses</code> , <code>type hints</code> , métodos estáticos y propiedades para la API interna.

2. Descripción del Flujo Implementado

VisioLab implementa un pipeline de procesamiento visual modular compuesto por cuatro etapas principales, cada una mapeada a un tab independiente en la interfaz. El flujo sigue el principio imagen → filtros → análisis / detección.

2.1 Ingesta de Imagen (Entrada)

El sistema acepta dos fuentes de entrada configurables desde el sidebar:

- Imagen del usuario: archivo subido vía file uploader (formatos JPG, PNG, BMP, WebP). El módulo `StreamlitBridge.bytes_to_bgr()` convierte los bytes crudos al formato NumPy BGR que usa OpenCV.
- Imagen sintética: generada programáticamente con formas geométricas, degradados y ruido controlado. Permite usar la app sin necesidad de una imagen propia.

- Imagen por webcam: usando el widget `streamlit camera_input` permite al usuario capturar una imagen por medio de su webcam y se repite el procesamiento igual que Imagen del usuario.

2.2 Tab 1 — Filtros y Transformaciones (`filters.py`)

El usuario construye un pipeline de filtros apilables desde el sidebar. Cada filtro se configura con parámetros ajustables mediante sliders y se agrega a una cola de procesamiento secuencial.

- **Gaussian Blur, Median Blur, Bilateral Filter, Box Blur.** Suavizado (Blur):
- **Sharpen (kernel laplaciano), Unsharp Mask.** Nitidez:
- **Canny, Sobel, Laplaciano.** Detección de bordes:
- **Erosión, Dilatación, Open, Close, Gradient, Tophat, Blackhat.** Morfología:
- **Contraste/brillo, gamma, saturación, matiz, temperatura de color, ecualización CLAHE/global.** Color y brillo:
- **Cartoon, Emboss, Sepia, Night Vision, Vignette, Pixelate.** Efectos especiales:

La clase `FilterPipeline.apply()` ejecuta los filtros activos en orden y retorna un `ProcessingResult` con la imagen resultante y metadata. El sidebar también expone 4 presets rápidos: Retrato, Noche, Artístico y Bordes.

2.3 Tab 2 — Detección de Objetos (`detection.py`)

Sobre la imagen filtrada (o la original si no hay filtros) se ejecuta inferencia con YOLOv8. El flujo es:

1. Carga lazy del modelo seleccionado (Nano / Small / Medium) en la primera inferencia.
2. `ObjectDetector.detect()` retorna un `DetectionResult` con la lista de objetos detectados, imagen anotada, tiempo de inferencia y conteo por clase.
3. `DetectionAnalytics.overlay_heatmap()` genera un mapa de calor de densidad de detecciones superpuesto a la imagen.
4. Se muestran gráficos de distribución por clase y distribución de confianza usando `ChartBuilder`.

2.4 Tab 3 — Análisis y Métricas (`analysis.py`)

Es el módulo central del trabajo de Dev 3. Compara la imagen original con la procesada y expone ocho visualizaciones:

- Métricas de calidad: MSE, PSNR, SSIM y diferencia media con etiqueta de calidad (Excelente / Buena / Aceptable / Degradada).
- Histogramas BGR comparativos con opción de escala logarítmica.
- Mapa de diferencias global y por canal individual (B/G/R) usando `colormaps` MAGMA/COOL/SUMMER/HOT.
- Tablas de estadísticas por canal: media, std, mín, mediana, máx y entropía de Shannon.
- Métricas de brillo perceptual (luminancia BT.601) y contraste RMS.
- Gráfico de percentiles P25/P50/P75 comparando original vs procesada.
- Gráfico radar de entropía de Shannon por canal (mide variedad tonal).
- Perfil de intensidad media horizontal/vertical para detectar gradientes de iluminación.
- Balance de color: donut con peso relativo de cada canal BGR.

- Inspección de matriz: tabla interactiva 20x20 de valores de píxeles por canal.

2.5 Tab 4 — Pipeline Completo

Integra los tres módulos anteriores en un único flujo secuencial ejecutado con un botón: filtros configurados → detección YOLO → histogramas comparativos y gráfico de clases detectadas. Permite validar el efecto de cada filtro sobre la capacidad de detección del modelo.

3. Dificultades Encontradas

Durante el desarrollo del proyecto se identificaron los siguientes retos técnicos y de integración, junto con las soluciones aplicadas:

Dificultad	Solución aplicada
Colisión de IDs en <code>st.plotly_chart()</code> : Streamlit genera IDs automáticos basados en el contenido de la figura. Al renderizar histogramas similares en el Tab 3 y Tab 4, los IDs coincidían y la app lanzaba un error en tiempo de ejecución.	Se agregó el parámetro <code>key=</code> único a todos los 12 llamados de <code>st.plotly_chart()</code> en el archivo, usando un prefijo por tab (<code>tab1_</code> , <code>tab2_</code> , <code>tab3_</code> , <code>tab4_</code>).
Compatibilidad de formatos BGR vs RGB: OpenCV trabaja internamente en BGR, mientras Streamlit y PIL esperan RGB. Las imágenes mostraban canales de color invertidos en la interfaz.	Se centralizó la conversión en <code>StreamlitBridge.bgr_to_rgb()</code> y se aplicó sistemáticamente antes de cada llamado a <code>st.image()</code> .
Carga del modelo YOLO en cada interacción: Streamlit re-ejecuta el script completo en cada acción del usuario, lo que provocaba que el modelo se recargara del disco en cada click, generando latencias de varios segundos.	Se implementó carga lazy con un atributo <code>_model</code> en <code>ObjectDetector</code> y se usó <code>st.session_state</code> para persistir el detector entre re-renders.
Imágenes en escala de grises en filtros de bordes: Algunos filtros (Canny, Sobel) retornan imágenes de 1 canal, mientras que el pipeline espera siempre 3 canales BGR. Esto causaba crashes en operaciones posteriores.	Los filtros de bordes en <code>SpatialFilters</code> se modificaron para retornar siempre BGR mediante <code>cv2.cvtColor(edges, COLOR_GRAY2BGR)</code> al final de cada método.
Entropía y estadísticas en imágenes convertidas a espacios no-BGR: Al cambiar el espacio de color (HSV, LAB, grises), <code>compute_extended_stats()</code> asumía 3 canales y fallaba.	Se agregó manejo explícito de canales en <code>ImageAnalyzer</code> : detección automática de número de canales, fallback a canal único para imágenes en grises y etiquetas dinámicas.
Dimensiones inconsistentes entre imagen original y procesada: Algunos filtros (<code>pixelate</code> , <code>resize</code> interno) podían alterar las dimensiones. MSE y SSIM requieren dimensiones idénticas.	Se agregó un guard de <code>cv2.resize()</code> al inicio de <code>compare()</code> y <code>compute_difference_map()</code> para normalizar siempre al tamaño de la imagen original antes de calcular métricas.

4. Scripts y Archivos del Proyecto

El proyecto está organizado en cuatro módulos Python independientes con responsabilidades bien delimitadas:

Archivo	Descripción
<code>demo.py</code>	Aplicación principal Streamlit. Define la UI completa: sidebar, tabs, estilos CSS y orquesta los cuatro módulos.
<code>filters.py</code>	Filtros espaciales y de color (Dev 1 / Dev 2). Clases <code>SpatialFilters</code> , <code>ColorAdjustments</code> , <code>SpecialEffects</code> y <code>FilterPipeline</code> .
<code>detection.py</code>	Detección de objetos con YOLOv8 (Dev 2). Clases <code>ObjectDetector</code> y <code>DetectionAnalytics</code> .
<code>analysis.py</code>	Análisis de imagen y métricas (Dev 3). Clases <code>ImageAnalyzer</code> , <code>MatrixExtractor</code> y <code>ChartBuilder</code> con soporte completo de histogramas dinámicos.
<code>requirements.txt</code>	Dependencias del proyecto: streamlit, opencv-python, numpy, plotly, Pillow, ultralytics, scikit-image.

Repositorio del proyecto:

GitHub: [\[Enlace al repositorio pendiente de agregar\]](#)